

## Task Queues and Action Selection

Task queues are the mechanism that turns agent responsibility into executable work. Rather than asking a section agent to infer its next move from a free-form conversation, the system gives it a durable, ordered list of declared tasks. Each queued task records an *action id*, a task context, a priority, a status, and the prompt or payload needed to perform the work. In practice, this means that a section agent can inspect its queue, identify which task corresponds to the current situation, and then execute that task without ambiguity.

The action id is the primary selector. It names the kind of work to be done, such as drafting a section, revising a section from feedback, responding to a callback, requesting research, or coordinating with a sibling agent. The task context narrows that action to the relevant section, claim, or artifact. For example, a task may point to a specific section id, a feedback message, a missing citation, or a diagram opportunity. The queue therefore separates *what* kind of work is needed from *where* that work applies.

Priority determines ordering when more than one task is available. Higher-priority tasks are chosen first, but only among tasks that are actually eligible to run. A section agent does not simply pick the oldest item in the queue; it evaluates the declared tasks against the current state of the section, the document, and any callback messages that have arrived. This keeps urgent repairs, explicit callbacks, and prerequisite work from being buried behind routine drafting tasks.

The prompt attached to a task is generated from the task context and the current document state. For a drafting task, the prompt may include the section title, summary, sibling section context, and the relevant outline path. For a revision task, it may include the feedback that triggered the revision and the exact constraints that must be preserved. For a research task, it may include the unsupported claim or missing definition that needs external support. The prompt is not a generic instruction; it is a task-specific working brief assembled from durable context.

Task status transitions make the queue observable. A task typically moves from queued to active when the agent begins work, and then to completed, failed, or blocked depending on the outcome. If a prerequisite is missing, the agent should not fabricate the missing material; instead, it should leave the task unresolved and queue or request the prerequisite work. This is especially important for citations, definitions, and file paths, where guessing would create hidden errors in the manuscript.

Section agents choose which declared task to run by matching the queue against the current situation. If the agent has been started manually, it first inspects the full document, its own section, the outline, registered references, and diagram opportunities, then uses that inspection to decide which task is the correct next action. If a callback message arrives, the callback is treated as work rather than as a passive notification. If the section needs a revision, the agent revises; if it needs research, the agent requests research; if it needs coordination, the agent responds to the sibling agent or hypervisor. The queue is therefore not just a list of jobs, but the runtime representation of editorial intent.

This selection process is constrained by declared task modes. A section agent should not invent a new action when an existing task already describes the needed work. It should also not ignore a callback in favor of a lower-value drafting task. The intended behavior is disciplined and local: inspect the queue, identify the task whose action id and context match the current state, and execute that task with the least possible drift.

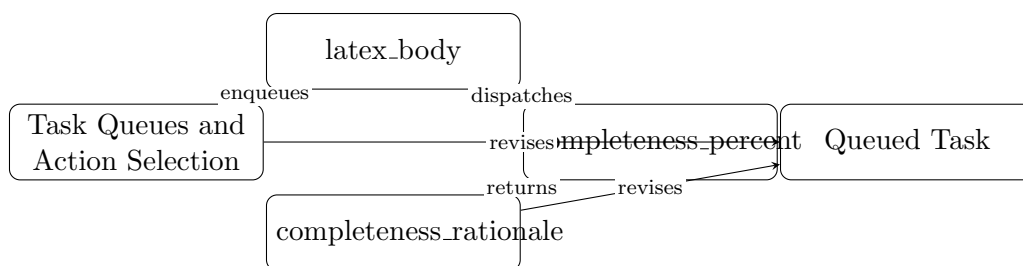
In the Codynamic Book Machine, this discipline matters because the manuscript is not edited by hidden side effects. Every meaningful change is routed through a task, every task has a status, and every status change can be inspected later. The result is a section workflow that is both autonomous and auditable: the agent can act on its own, but it does so by selecting from explicit

work items rather than by improvising outside the queue.

A concrete way to read this section is as a decision rule for the section agent. If the queue contains a direct drafting task for the current section, that task is the default choice. If a callback has arrived, the callback takes precedence because it represents a concrete unresolved event. If the section is missing a prerequisite, such as a citation key or a diagram path, the agent should not continue as if the prerequisite existed; it should route the missing work to the appropriate specialist workflow and mark the current task as blocked or pending. That ordering keeps the runtime honest about what can actually be completed now.

The same rule also explains why the queue is durable rather than conversational. A chat exchange can suggest what should happen next, but the queue records what the system has formally committed to do. That distinction lets the hypervisor, the section agent, and later maintenance passes inspect the same work list and agree on which action is active, which one is waiting, and which one has already been resolved. In other words, the queue is the manuscript’s executable to-do list, not just a memory aid.

For this chapter, the task-queue model also provides the bridge to the next section on messages and callbacks. Tasks say what should happen; messages say what was said; callbacks convert message events into new work. Together, they form the runtime loop that keeps the manuscript moving without losing track of responsibility.



Conceptual diagram for 'Task Queues and Action Selection': Explain queued tasks, action ids, task context, priorities, generated prompts, status transitions, and how section agents choose which declared task to run.

Figure 1: Conceptual diagram for task queues and action selection.